PROJECT REPORT

Personalization (Context-Aware Agent)

Submitted under Samsung PRISM Web-Agent Hackathon

KONTA

Understands what you're doing. Helps when it matters.

Submitted by:
Team Name: Aegis
Team Members:
Anoushka Awasthi (aawasthi_be23@thapar.edu)
Aahil Khan (akhan_be23@thapar.edu)
Anshum Singla (asingla5_be23@thapar.edu)
Sanyam Wadhwa (swadhwa1_be23@thapar.edu)
Institution:
Thapar Institute of Engineering and Technology

Table of Contents

Executive Summary 3

Problem Statement 4

Architecture 6

System Design 12

Use-Cases 20

- Use Case 1: Research Student Managing Multi-Day Academic Projects 20
- Use Case 2: Web Developer Managing Multiple Client Engagements 20
- Use Case 3: Content Creator Managing Research-Heavy Production Pipelines 21
- Use Case 4: Privacy-Conscious User Organizing Personal Browsing Locally 21

External Dependencies 23

1. Core Framework Dependencies 23
2. Machine Learning Dependencies 23
3. Visualization Dependencies 23
4. Chrome APIs (Built-in Dependencies) 24
5. Styling Dependencies 24
6. Development Dependencies 25

Impact 26

- User Impact 26
- Technical Impact 26
- Research Impact 26
- Educational Impact 27

Challenges & Limitations 28

- Technical Challenges 28
- Privacy & Security Limitations 30
- Scalability Limitations 31
- User Experience Challenges 31
- Compatibility Limitations 32

Key Learnings 33

- Technical Learnings 33
- Design Learnings 34
- User Research Learnings 34
- ML Learnings 35
- Product Learnings 35
- Privacy & Ethics Learnings 36

Conclusion 37

Executive Summary

Konta is a privacy focused Chrome extension that converts everyday web browsing into a structured and intelligent knowledge management system. It addresses a common and persistent challenge faced by most users: information discovered online is quickly forgotten, scattered across tabs, or buried in unsearchable browser history. Traditional tools such as bookmarks require manual effort and discipline, browser history lacks meaningful organization, and cloud-based AI tools raise serious concerns around data ownership and privacy.

Konta works quietly in the background, capturing browsing activity and transforming it into a coherent personal knowledge base without interrupting the user's workflow. Instead of treating each visited page as an isolated item, Konta preserves context and continuity across browsing sessions. It enables users to revisit previous research, recall why certain information mattered, and understand how their interests and ideas evolved over time.

The system achieves this by applying machine learning techniques directly within the browser. Semantic representations of web content are generated locally, allowing related pages to be grouped and connected based on meaning rather than simple keywords or timestamps. These relationships are then presented through an interactive knowledge graph that helps users explore their browsing history in a visual and intuitive manner, revealing patterns and connections that would otherwise remain hidden.

A defining characteristic of Konta is its fully local architecture. All machine learning inference, data processing, and analysis are executed on the user's device using browser compatible technologies such as Web Assembly. No data is sent to external servers, no cloud storage is involved, and no third-party APIs are required. This approach ensures complete privacy, minimizes latency, and always gives users full control over their data.

By combining passive data capture, local machine learning, and meaningful visualization, Konta reframes browsing history as an active cognitive asset rather than a disposable trail of links. It empowers users to build a personal, private, and continuously evolving knowledge system directly from how they naturally explore the web.

Problem Statement

The Challenge: Knowledge is Captured, But Not Retained

Modern web browsers enable us to consume vast amounts of information, yet we lack effective tools to organize, recall, and connect what we have learned. Existing solutions each address parts of the problem but fall short in critical ways.

Current Solutions & Their Limitations

1. **Chrome Bookmarks**
   What It Does: Manual folder-based organization of saved pages
   Limitations: -
   - Fragmented Memory: Only captures explicitly bookmarked pages (typically <5% of browsing)
   - No Semantic Recall: Search by exact title/URL only not by meaning
   - Manual Organization Burden: Requires creating folder structures, choosing categories
   - No Context: Loses temporal relationships, research flow, session boundaries
   - No Discovery: Cannot surface connections between bookmarked pages
   - Result: Bookmarks become a graveyard of hundreds of unorganized links that users never revisit

2. **Browser History**
   What It Does: Chronological log of all visited URLs
   Limitations: -
   - No Organization: Flat list with no grouping, no relationships
   - No Semantic Recall: Search by URL/title text only; fails on conceptual queries
   - Overwhelming Volume: Thousands of entries with no filtering by importance
   - No Privacy: Some browsers sync history to cloud servers
   - No Metadata: Does not capture what you learned, just what you clicked
   - Result: History is useful for “I visited this yesterday” but useless for “What did I read about X last month?”

3. **Note-Taking Apps (Notion, Obsidian, Roam)**
   What They Do: Manual knowledge management with wikis, backlinks, databases
   Limitations: -
   - Manual Organization Burden: Requires actively taking notes, creating pages, linking concepts
   - Fragmented Memory: Only captures what you explicitly write down (~10% of what you read)
   - Context Switching: Must leave browser, open app, decide where to save info
   - Privacy Trade-offs (Notion): Cloud-hosted, requires account, data on external servers
   - Setup Complexity: Steep learning curve, template configuration, plugin management
   - Result: Works for dedicated note-takers, but most browsing never makes it into the system

4. **Cloud-Based “AI Research Tools” (Mem, Recall, Fabric)**
   What They Do: Automatic capture with AI-powered search and organization
   Limitations: -
   - Privacy Trade-offs: Send entire browsing history to external servers
   - Subscription Required: $10-30/month for processing, storage, and API costs
   - Data Lock-In: Owned by company; if service shuts down, data is lost
     Latency: Round-trip to cloud adds 500ms-2s per search
   - Internet Required: Does not work offline
   - Trust Issues: Users uncomfortable with companies analysing sensitive browsing
   - Result: Powerful but incompatible with privacy-conscious users and offline scenarios

The Core Problems

Existing solutions fail to address four fundamental challenges:

1. **Fragmented Memory Problem**: Information scattered across bookmarks (manual), history (unorganized), notes (incomplete), and forgotten tabs
   Impact: - Cannot remember which article explained a concept - Re-Google the same questions repeatedly - Lose track of multi-day research threads
   What's Needed: Automatic, comprehensive capture of all browsing with intelligent organization

2. **No Semantic Recall Problem**: Can only search by exact words in titles/URLs; cannot search by meaning or concept
   Examples: - Search "authentication" → Misses page titled "Login Implementation Guide" - Search "async patterns" → Misses "Promises and Async/Await Tutorial" - Search "graph databases" → Misses "Neo4j Beginner's Guide"
   Impact: Must remember exact phrasing to find what you saved
   What's Needed: Semantic search that understands synonyms, related concepts, and context

3. **Privacy Trade-offs Problem**: Powerful tools require sending browsing history to external servers
   Concerns: - Medical research → Health data exposure - Job hunting → Current employer could discover - Financial research → Trading strategies visible to third parties - Personal interests → Profiling and targeting
   Impact: Privacy-conscious users forced to choose between functionality and data control
   What's Needed: Local processing with no external data transmission

4. **Manual Organization Burden Problem**: Effective organization requires constant manual effort
   Cognitive Load: - Deciding what to save (bookmark vs ignore) - Choosing which folder/category fits - Creating tags and labels - Writing notes and summaries
   Reality: Users do not have time/discipline to organize while researching
   Impact: Either no organization (chaos) or abandoned organization systems(friction)
   What's Needed: Automatic organization that learns from behaviour, not explicit input

What Success Looks Like

An ideal solution would:

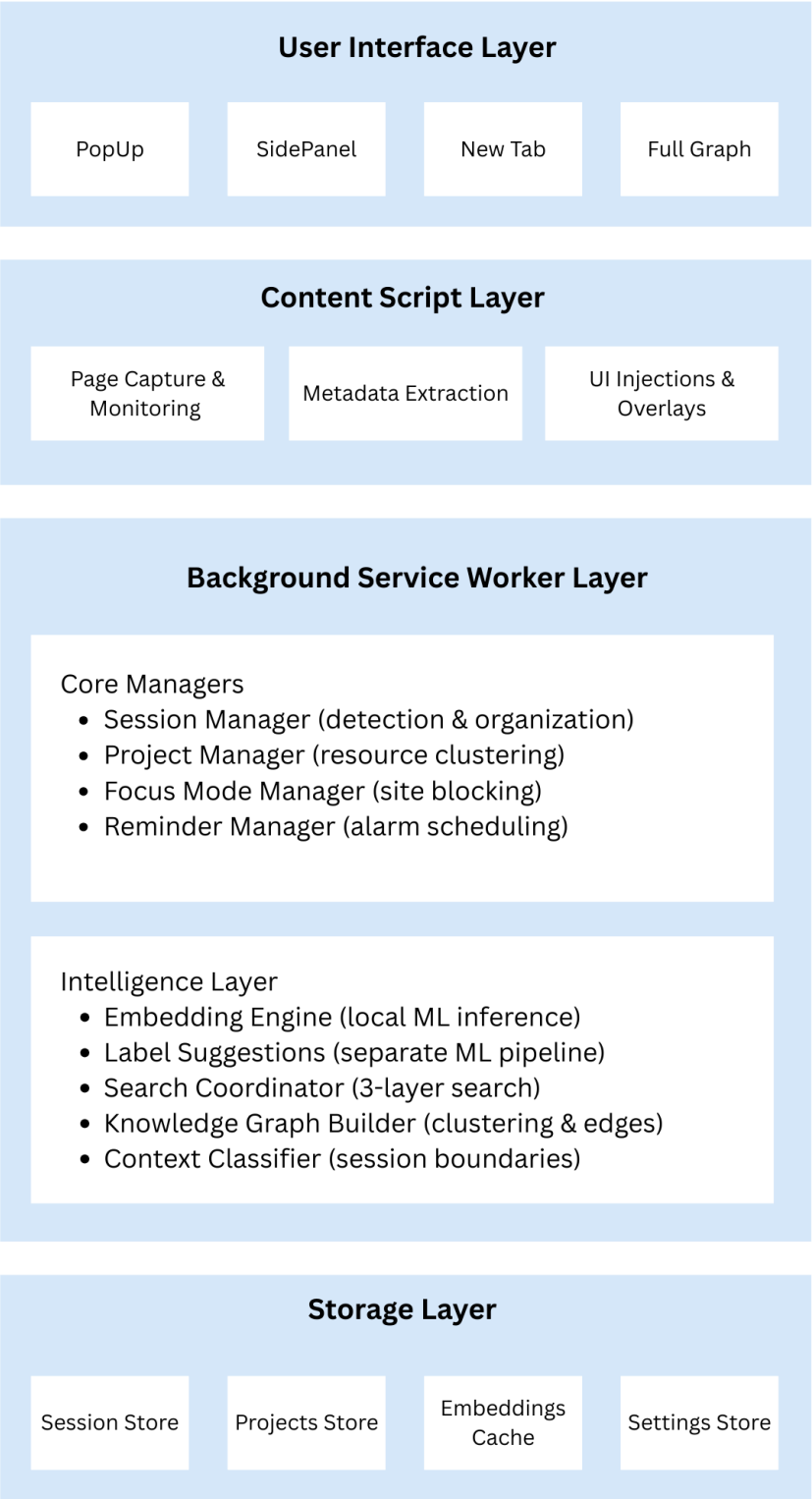
1. **Automatically capture** all browsing without manual bookmarking
2. **Organize intelligently** using sessions, projects, and temporal context
3. **Enable semantic search** to find pages by meaning, not just keywords
4. **Visualize connections** between related information
5. **Process locally** with no cloud dependency or privacy compromise
6. **Work passively** with minimal user input required

Konta is designed to deliver on all six criteria.

Architecture

High-Level Overview

Konta follows a **four-layer architecture** designed for Chrome Extension Manifest V3:



Component Responsibilities

1. User Interface Layer

- **Popup**: Quick actions, session overview, settings access
- **Sidepanel**: Full-featured workspace with timeline, graph, projects, search
- **New Tab**: Welcome page with consent screen
- **Full Graph Page**: Immersive graph visualization with advanced filtering

2. Content Script Layer

- **Page Capture**: Monitors DOM, extracts metadata, tracks user engagement
- **Scroll/Dwell Tracking**: Measures actual page engagement
- **UI Injections**: Adds notification indicators, quick-add buttons
- **Search Enhancement**: Extracts search queries from Google results

3. Background Service Worker

- **Event-Driven Architecture**: Responds to page visits, alarms, user actions
- **Stateless Design**: All state persisted to Chrome Storage
- **Message-Based Communication**: chrome.runtime.sendMessage for cross-component messaging

4. Storage Layer

- **Chrome Storage API**: ~10MB quota for extensions
- **Structured Data**: JSON-serialized objects with timestamps
- **Caching Strategy**: Embeddings cached indefinitely, sessions pruned by age

System Walkthrough: End-to-End Flow

This section provides a complete walkthrough of how Konta works from the user's perspective, tracing data flow through all system layers.

Step-by-Step Journey

1. User Opens Chrome and Browses

```
User Action: Opens new tab → navigates to github.com/react/react
```

What Happens: - Content script injected into page - Monitors DOM for metadata (title, description, keywords) - Tracks scroll depth and dwell time (engagement signals)

2. Page Event Captured

```
Content Script → Background Service Worker
Message: { type: "PAGE_VISIT", url: "github.com/react/react", title: "React - A JavaScript library..." }
```

What Happens: - Background worker receives page visit event - Extracts domain, timestamp, search query (if from Google) - Triggers session detection algorithm

3. Session Detection

```
Background Worker: sessionManager.handlePageVisit()
- Loads last session from Chrome Storage
- Checks: Was last visit < 30 min ago?
  → YES: Same session (research continues)
  → NO: New session (new research thread)
- Checks: Has session duration exceeded 2 hours?
  → YES: Split into new session
- Checks: Is page context different from last page?
  → If YES + 5min gap: New session
```

Decision: Add page to existing "React Learning" session

Storage Update:

```
{
    "sessionId": "abc-123",
    "startTime": 1704700800000,
    "pages": [
      { "url": "github.com/react/react",  "title": "React...",  "timestamp": 1704700900000 }
    ]
}
```

4. Embedding Generation (Async)

```
Background Worker: embeddingEngine.generateEmbedding()
- Extracts text: "React A JavaScript library for building user interfaces"
- Loads ML model (Transformers.js + WASM)
- Generates 384-dimensional vector: [0.12, -0.34, 0.56, ...]
- Inference time: ~80ms
```

Storage Update:

```
{
    "embeddings":  {
      "github.com/react/react": [0.12, -0.34, 0.56, ...]
    }
}
```

Why This Matters: Enables semantic search (find by meaning, not just keywords)

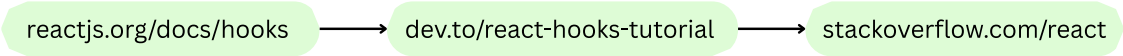
5. Knowledge Graph Update

```
Background Worker: knowledgeGraphBuilder.addNode()
- Creates new node for this page
- Computes edges to existing pages:
  - "reactjs.org/docs/hooks" → similarity 0.82 (thick edge)
  - "vuejs.org" → similarity 0.41 (thin edge)
  - "python.org" → similarity 0.12 (no edge, below threshold)
- Runs clustering algorithm (Louvain)
- Updates cluster: "React Ecosystem" (8 pages)
```

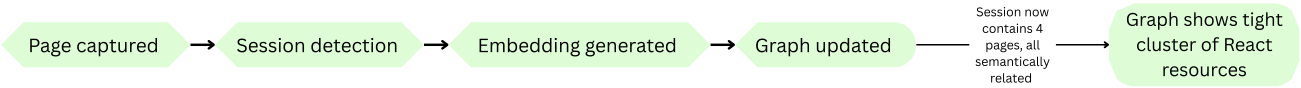
What User Sees: New node appears in graph, connected to related React pages

6. User Continues Browsing

User visits:



What Happens (repeats for each page):



7. Project Detection (Background)

```
Background Worker: projectManager.detectCandidates()
- Runs every 10 page visits (triggered)
- Analyzes all sessions for patterns:
  - "React" appears in 3 different sessions
  - 12 pages across 5 domains
  - Duration: 6 hours over 2 days
  - Score: 75 points → Qualifies as project
- Creates project candidate: "React Learning"
```

What User Sees: Notification badge → “New project detected: React Learning”

8. User Searches Later

```
User Action (1 week later): Opens sidepanel → searches "hooks tutorial"
```

What Happens:

```
Search Coordinator:
1. Layer 3 (ML Ranker): No learned associations yet → skip
2. Layer 2 (Semantic Search):
   - Generate query embedding: [0.15, -0.31, 0.58, ...]
   - Compare with all 150 cached embeddings
   - Cosine similarity scores:
     * dev.to/react-hooks-tutorial → 0.89
     * reactjs.org/docs/hooks → 0.84
     * stackoverflow.com/react-hooks → 0.76
   - Return top 10 results ranked by similarity
```

What User Sees: Results appear in <300ms, including pages that don’t contain exact words “hooks” or “tutorial” in title (semantic matching)

9. User Explores Graph

```
User Action: Clicks "Graph" tab in sidepanel
```

What Happens:

```
- Load all pages from storage (150 pages)
- Filter by time range (last 3 months)
- Render force-directed graph:
- Nodes = Pages (coloured by cluster)
- Edges = Similarity (thick = strong, thin = weak)
- Clusters = Topics (React, Python, News, etc.)
- Physics simulation runs (spring forces, repulsion)
```

What User Sees:

- Visual clusters of related content
- “React Ecosystem” cluster (8 nodes, tightly connected)
- Discovers forgotten connection: “React” -> “TypeScript” edge
- Hover over node -> Shows page title, domain, visit count

10. User Sets Reminder

```
User Action: Right-clicks project → "Set Reminder" → "Weekly review"
```

What Happens:

```
Background Worker: reminderManager.createReminder()
- Creates alarm: Every Monday 9 AM
- Stores: { projectId, frequency: "weekly", time: "09:00" }
```

Next Monday 9 AM:

```
Chrome Alarm Fires → Background Worker → Create notification
User sees: "Time to review React Learning project (12 resources)"
```

11. Focus Mode Activation

```
User Action: Enables "Study Mode" focus preset
```

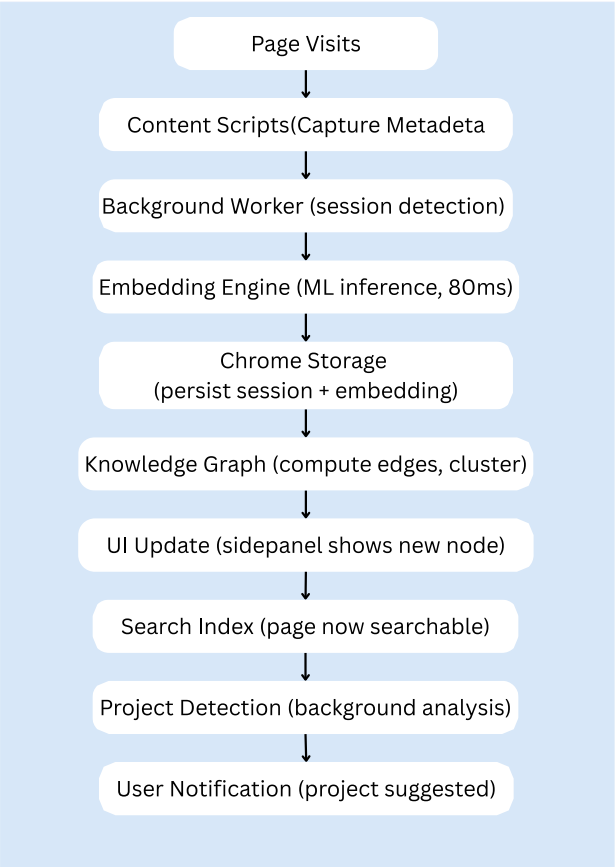
What Happens:

```
Background Worker: focusModeManager.activateMode()
- Reads preset rules: Block social media, news, entertainment
- Uses chrome.declarativeNetRequest API
- Creates blocking rules:
  - twitter.com → redirect to motivation page
  - reddit.com → redirect to motivation page
  - youtube.com → redirect to motivation page
```

User tries to visit Reddit:

```
Request intercepted → Blocked → Shows:
"Focus Mode Active: Stay focused on your goals!"
```

Data Flow Summary



Typical Timing

1. Page capture: 20-50ms (imperceptible)
2. Session detection: 5-10ms (fast)
3. Embedding generation: 80ms (async, does not block)
4. Storage writes: 20-40ms (async)
5. Graph update: 100-200ms (runs in background)

Total user-facing latency: <100ms (page capture only)

Why This Architecture Works

1. **Automatic Capture**: User does not manually save anything
2. **Smart Organization**: Sessions and projects detected from behaviour
3. **Semantic Understanding**: ML embeddings enable concept-based search
4. **Visual Discovery**: Graph reveals connections user did not notice
5. **Privacy-Preserving**: Everything happens locally, no external APIs
6. **Performance**: Background processing keeps UI responsive

User Benefit: Effortless knowledge management that learns from your browsing

System Design

1. Session Detection Algorithm

Purpose: Automatically group related browsing activity into logical sessions
Criteria for Session Boundaries:
- **Inactivity Gap**: 30+ minutes idle → new session
- **Maximum Duration**: 2+ hours continuous → new session
- **Context Switch**: 5+ minute gap + different page context → new session

```
Contexts: work | shopping | entertainment | research | social | news | other

Classification based on:
- URL patterns (github.com → work, amazon.com → shopping)
- Page title keywords
- Domain categories
```

Implementation Flow & Data Structure:

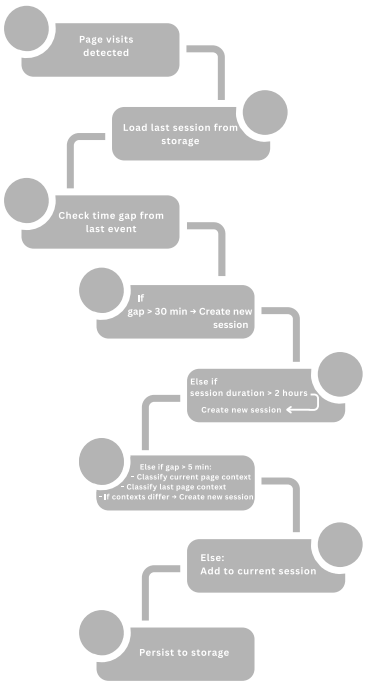


Fig: Implementation Flow

```
Session {
        id: string (UUID)
        startTime: number (timestamp)
        endTime: number (timestamp)
        pages: PageEvent[]
        inferredTitle: string (from dominant keywords)
        label?: string (user-assigned)
        dominantLabel?: string (auto-detected context)}


PageEvent {
        url: string
        title: string
        domain: string
        timestamp: number
        scrollDepth: number (0-100)
        dwellTime: number (milliseconds)
        visitCount: number (revisits in session)
        searchQuery?: string (if from search result)
        embedding?: number[384] (ML vector)
    }
```

Fig: Data Structure

2. Project Detection System

Purpose: Identify cohesive research topics across multiple sessions Algorithm:

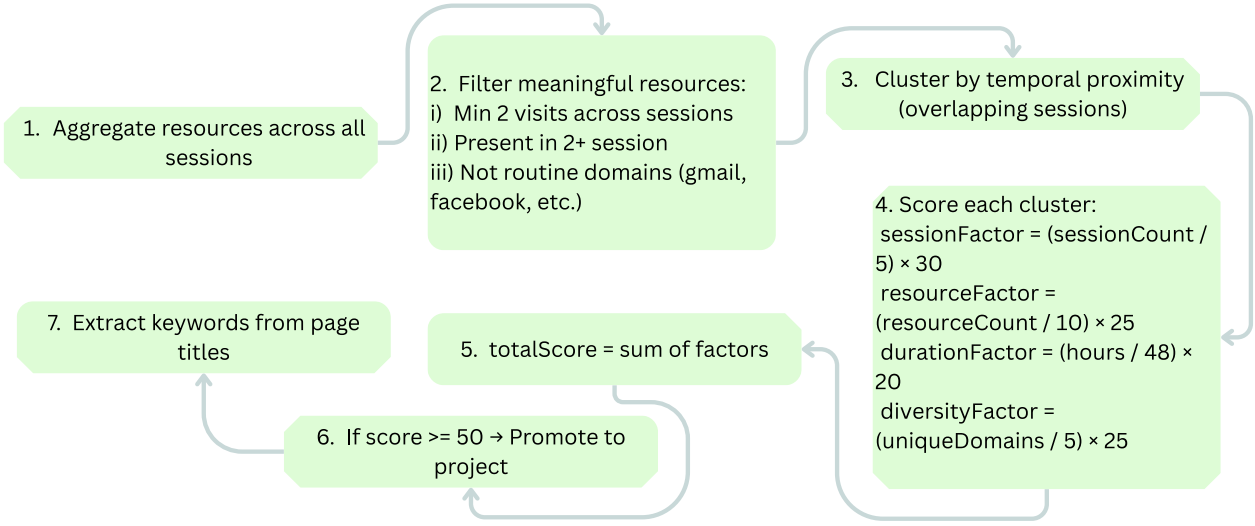


Fig: Multiple sessions Algorithm

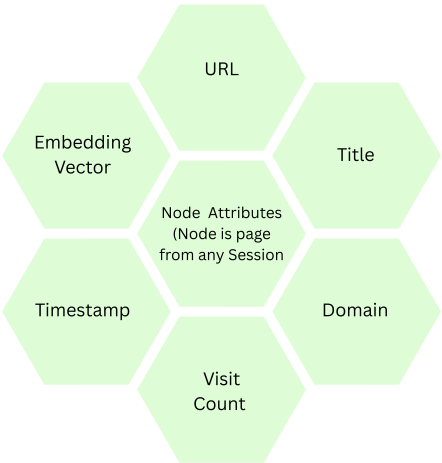
Generate project name from top keywords

Scoring Example:

```
Project: "React Hooks Documentation"
- Sessions: 4 → (4/5) × 30 = 24 points
- Resources: 12 → (12/10) × 25 = 25 points (capped)
- Duration: 36 hours → (36/48) × 20 = 15 points
- Domains: 3 → (3/5) × 25 = 15 points
Total: 79 points → Qualifies as project
```

3. Knowledge Graph Construction

Purpose: Visualize relationships between pages and identify topic clusters




```
For each pair of nodes:
  if both have embeddings:
    embeddingSim = cosineSimilarity(vec1, vec2)
    keywordSim = jaccardSimilarity(keywords1, keywords2)
    temporalProximity = exp(-timeDelta / threshold)

    domainBoost = (sameDomain ? 1.2 : 1.0)

    compositeWeight = (
      embeddingSim × 0.5 +
      keywordSim × 0.3 +
      temporalProximity × 0.2
    ) × domainBoost

    if compositeWeight > 0.3:
      createEdge(node1, node2, compositeWeight)
```

Fig: Node representation and edge computation

Manual Links as Ground Truth:
Users can create manual edges by using the link option in the graph interface. These manual links are treated as ground truths and receive special handling:

- **Permanent Weight**: Manual edges assigned maximum weight (1.0) regardless of semantic similarity
- **Persist Across Recalculations**: Automatic edge recompilation never removes manual edges
- **Visual Distinction**: Displayed with distinct styling (e.g., dashed lines or different colour)
- **ML Training Signal**: Used to improve Layer 3 ML Ranker by teaching the system which connections matter to the user
- **Override Threshold**: Manual edges bypass the 0.3 similarity threshold requirement

This approach combines automatic discovery with user expertise, allowing users to correct false negatives (missed connections) and reinforce important relationships the algorithm might underweight.

Clustering:
- **Algorithm**: Louvain community detection
- **Purpose**: Group related pages into topics/themes
- **Label Generation**: Most common domain or keyword in cluster
- **Visualization**: Color-coded clusters with dashed boundaries

Understanding the Knowledge Graph:
Visual Legend The knowledge graph is a force-directed visualization that reveals patterns in your browsing. Here is how to interpret what you see:

Nodes (Circles) What They Represent:

Each circle = one webpage you have visited

Visual Attributes:
- **Size**: Larger nodes = more visits or longer dwell time (important pages)
- **Colour**: Cluster membership
- **Position**: Determined by similarity (related pages pulled together)

Interaction: -
Hover → Shows page title, domain, visit count, timestamp
Click → Opens page details panel or navigates to URL –
Drag → Pin node in place (disable physics)

Edges (Lines) What They Represent:

Each line = semantic relationship between two pages

What Determines Similarity: - 50% Semantic embeddings (ML-based meaning) - 30% Keyword overlap (shared terms) - 20% Temporal proximity (visited close in time)

Cognitive Interpretation:
- **Normal edges** → "These pages are related to each other"
- **Dashed edges** → "I explicitly connected these" (manual links)
- **Multiple paths** → "These pages form a learning progression"

Clusters (Coloured Groups)
What They Represent: Automatically detected topics or research themes
Visual Attributes:
- **Dashed boundaries**: Enclose nodes in same cluster
- **Colour coding**: Each cluster gets unique color
- **Label**: Displays dominant keyword or domain (e.g., "React Ecosystem", "Machine Learning")

Clustering Algorithm: Louvain community detection
- Analyses edge weights to find densely connected groups
- Optimizes for high internal connectivity, low external connectivity
- Typical result: 5-15 clusters for 100 pages

Cognitive Interpretation:
- **Tight cluster** (nodes close together) → Focused research on single topic
- **Loose cluster** (nodes spread apart) → Broad exploration
- **Bridge nodes** (between clusters) → Pages connecting multiple topics
- **Isolated nodes** → Unique pages without clear topic fit
Example Clusters:
- "Python ML" (8 pages): scikit-learn, TensorFlow, NumPy docs
- "Web Dev" (12 pages): React, Next.js, Tailwind tutorials "News" (15 pages): Various news articles (weak connections)

Spatial Distance
What it Means: Physical distance between nodes reflects

Distance	Relationship	Interpretation
Very Close	High Similarity (same cluster)	Pages about some narrow topic
Near	Moderate SImilarity	Pages about related topics
Far	Low Similarity	Pages about unrelated topics
Opposite Sides	No similarity	Completely distinct research areas

Table: Interpretation of Physical Distance

Why Nodes Move: Force-directed physics simulation
- **Attraction**: Edges pull connected nodes together (spring force)
- **Repulsion**: All nodes push away from each other (charge force)
- **Collision**: Nodes avoid overlapping (collision detection)
- **Centre Gravity**: Weak pull toward centre (prevents drift)

Equilibrium: Nodes settle where forces balance

- Strong edges → Nodes pulled very close
- Weak edges → Nodes remain distant
- No edge → Only repulsion (push apart)

Cognitive Interpretation: -

- “If I’m reading this page, what else should I read?” → Look at nearby nodes
- “What topics have I explored?” → Look at distinct spatial regions –
- “How does X relate to Y?” → Trace path through edges

Visual Patterns & What They Mean

Pattern	What it looks like	What it means
Dense Cluster	Many Nodes are tightly packed	Deep research on focused topic
Star Pattern	One central node with many edges	Hub page (e.g., documentation index)
Linear Chain	Nodes in a line	Learning progression or tutorial sequence
Scatter Nodes	Nodes with few Connections	Exploratory browsing, no clear focus
Bridge Nodes	Nodes between teo clusters	Page connecting different topics
Isolated Nodes	No edges to other nodes	Unique page, doesn’t fit any topic

Table: Interpretation of Visual Patterns

Example Interpretation:

```
Visual:
Dense "React" cluster (10 nodes) + Isolated "Cooking" node
Meaning:
User focused on React learning, with occasional unrelated browsing

Visual:
Bridge node "TypeScript" connecting "React" and "Node.js" clusters
Meaning:
TypeScript is shared technology between frontend (React) and backend (Node.js)

Visual:
Linear chain of nodes: "Intro" → "Intermediate" → "Advanced"
Meaning:
User followed structured learning path (tutorial series)
```

Interactive Features Zoom & Pan:

- Scroll to zoom in/out
- Drag background to pan
- Helps explore large graphs (100+ nodes)

Filtering (Sidepanel Controls):
- **Time Range**: Show only pages from last week/month/year
- **Domain**: Hide specific domains (e.g., exclude news sites)
-**Cluster**: Show only selected cluster
- **Similarity Threshold**: Adjust minimum edge weight (reduce clutter)

Search Highlight:
- Search for term → Matching nodes highlighted
- Dim non-matching nodes → Focus attention

Summary: The graph is a **spatial memory aid**, distance and connections mirror how concepts relate in your mind.

Use it to:

- **Rediscover**: "What did I read about X?"

- **Connect**: "How does X relate to Y?"

- **Explore**: "What else is in this topic?"

- **Reflect**: "What have I been learning?"

4. Multi-Layer Search System

Purpose: Provide intelligent search that learns from user behaviour

Layer 3: ML Ranker (Highest Priority)
- Activated when user has labelled sessions/projects
- Learns associations between queries and content
- Uses learned patterns to predict relevance
- Falls back to Layer 2 if no learned data

Layer 2: Semantic Search
- Generates query embedding using ML model
- Compares against all page embeddings
- Uses cosine similarity for ranking
- Threshold: 0.3 minimum similarity
- Falls back to Layer 1 if model not loaded

Layer 1: Keyword Search (Fallback)
- Traditional TF-IDF algorithm Tokenizes query and documents
- Ranks by term frequency × inverse document frequency
- Always returns results (guaranteed fallback)

Search Flow:

```
Query: "machine learning tutorials"

Layer 3 Check:
  → No learned associations yet
  → Skip to Layer 2

Layer 2 (Semantic):
  → Generate embedding: [0.12, -0.34, 0.56, ...]
  → Compare with 150 cached page embeddings
  → Find 12 pages with similarity > 0.3
  → Rank: pytorch.org/tutorials (0.87), tensorflow.org (0.79), ...
  → Return top results

If Layer 2 failed:
  → Layer 1 (Keyword):
    → Extract terms: ["machine", "learning", "tutorials"]
    → Calculate TF-IDF scores
`   → Return ranked results
```

5. Local ML Pipeline

Model: Xenova/all-MiniLM-L6-v2(sentence-transformers)

- **Type**: Transformer-based sentence encoder
- **Output**: 384-dimensional embeddings
- **Size**: ~23MB (ONNX format)
- **Backend**: WASM (Web Assembly) only (required for Manifest V3)

Initialization:

```
// Disable GPU/WebGL (not allowed in service workers)
env.backends.onnx.wasm.simd = false
env.backends.onnx.wasm.proxy = false
env.backends.onnx.webgpu = false
// Load model (first load downloads, subsequent loads use cache)
pipeline = await transformers.pipeline(
        'feature-extraction',
        'Xenova/all-MiniLM-L6-v2',
        { quantized: true }
)
```

Inference Process:

```
1. Page visit event received
2. Extract text: title + domain + keywords
3. Normalize: lowercase, trim whitespace
4. Check cache: if embedding exists, use it
5. Else:
   a. Pass text to model
   b. Apply mean pooling to output
   c. Normalize to unit vector
   d. Store 384-dim vector in cache
6. Trigger graph rebuild with new embedding
```

Performance: -

- First inference: ~500ms (model warmup)
- Subsequent inferences: ~50-100ms per page
- Cache hit: <1ms (storage lookup)
- Memory: ~80MB total (model + runtime)

Use Cases

Use Case 1: Research Student Managing Multi-Day Academic Projects

User Profile
A PhD research student working simultaneously on multiple academic projects, frequently revisiting papers, repositories, and documentation over extended periods.

Workflow
Upon installation, Konta imports recent browsing history and automatically organizes activity into context-aware browsing sessions. The Timeline view presents sessions chronologically, while topic-based clusters allow navigation across related research areas. Sessions can be manually labelled (e.g., Thesis Research, Teaching, Personal), and the system progressively auto-suggests labels based on usage patterns.

The Knowledge Graph visualizes semantic relationships between visited pages using locally generated embeddings, enabling concept-level recall rather than keyword-based search. Time-based filters allow focused exploration of recent or historical research.

Konta detects repeated visits to project-specific resources (e.g., GitHub repositories, documentation sites) across multiple sessions and automatically suggests project creation. Accepted projects can be expanded with related resources and reopened instantly as grouped tabs via reminders or contextual prompts.

Value Delivered

- Efficient rediscovery of previously read academic material
- Clear separation of concurrent research contexts
- One-click resumption of complex, multi-tab research projects
- Reduced cognitive overload through session segmentation and break prompts
- Full privacy through local-only processing

Use Case 2: Web Developer Managing Multiple Client Engagements

User Profile
A freelance web developer handling multiple client projects, each requiring frequent context switching across development, design, communication, and deployment tools.

Workflow
Projects can be created manually by grouping relevant resources such as repositories, design files, staging environments, and task boards. Scheduled reminders allow automatic loading of complete project contexts at predefined times.

Konta detects emerging projects by observing repeated visits to core project resources over multiple sessions and suggests project creation when relevance thresholds are met. Switching between client contexts is achieved by simply visiting a project's main resource, triggering instant loading of all associated sites.

The Knowledge Graph preserves the developer's problem-solving trail, clustering related documentation, issue threads, and solutions. Session labelling enables retrospective analysis of time spent per client or activity type.

Value Delivered

- Instant context switching between client projects
- Elimination of manual tab and bookmark management
- Automatic detection of new client engagements
- Improved accountability via session-based work review
- Faster recall of previously tested solutions

Use Case 3: Content Creator Managing Research-Heavy Production Pipelines

User Profile

A technology content creator producing long-form video essays requiring extensive research, synthesis, and production coordination.

Workflow

During the research phase, Konta automatically segments browsing into coherent sessions and topic-based clusters. The Knowledge Graph consolidates diverse sources—repositories, forums, papers, and articles—into semantically meaningful groups that directly inform content structure.

Clusters are reused during script writing via the Timeline's cluster view, enabling focused review of all material related to a specific section. A dedicated project is created to manage production assets such as scripts, storyboards, demo repositories, and publishing tools.

Quick actions allow on-the-fly addition of new references during filming without disrupting workflow. Session labels distinguish between research, writing, and administrative tasks for post-production review.

Value Delivered

- Automatic organization of large research volumes
- Clear mapping between research clusters and content structure
- Rapid access to references during production
- Insight into time allocation across creative phases
- Protection of sensitive research sources through exclusion controls

Use Case 4: Privacy-Conscious User Organizing Personal Browsing Locally

User Profile

A privacy-aware user seeking structured browsing organization without cloud storage, external analytics, or data sharing.

Workflow

During onboarding, Konta transparently communicates its local-only data handling model. Users can exclude sensitive domains (e.g., banking, medical, internal work tools) from all tracking and processing.

Non-sensitive browsing is organized into sessions and clusters visible in the Timeline and Knowledge Graph. Session labels support personal life organization without third-party monitoring. Long-term personal research (e.g., home improvement, financial planning) is preserved as reusable clusters or projects.

Focus Mode allows temporary blocking of distracting site categories, while notification settings provide granular control over system prompts. All data can be exported or imported locally, ensuring complete data ownership.

Value Delivered

- Full control over what is tracked and stored
- Local-first knowledge organization without surveillance
- Long-term recall of personal research without bookmarks
- Distraction management without behavioural profiling
- Transparent and auditable system behaviour

External Dependencies

1. Core Framework Dependencies

Plasmo Framework (v0.90.5)
Purpose: Chrome extension development framework
Why: Simplifies Manifest V3 development, provides hot reload, handles bundling
Type: Build-time dependency
Alternatives Considered: Raw Chrome Extension API (too verbose), Vite (less extension focused)
Risk: Framework updates may introduce breaking changes
Mitigation: Version locked, comprehensive testing before upgrades

React (v18.2.0) & React-DOM
Purpose: UI component framework
Why: Component reusability, declarative UI, large ecosystem
Type: Runtime dependency
Size: ~140KB (gzipped)
Alternatives Considered: Vue (less TypeScript support), Vanilla JS (too much boilerplate)

TypeScript (v5.3.3)
Purpose: Type safety, improved DX
Why: Catches bugs at compile time, better IDE support, self-documenting code
Type: Build-time dependency
Impact: +15% initial learning curve, -70% runtime type errors

2. Machine Learning Dependencies

Transformers.js (@xenova/transformers v2.17.2)
Purpose: Run ML models in browser via ONNX Runtime
Why: Only library supporting Transformers in browser with WASM
Type: Runtime dependency
Size: ~300KB library + 23MB model (first load only)
Critical: Entire ML functionality depends on this
Risk: Limited model support, WASM performance constraints
Alternatives: None viable for local browser ML with Manifest V3

ONNX Runtime Web (v1.14.0)
Purpose: Execute ONNX models via Web Assembly
Why: Required by Transformers.js, Manifest V3 compatible
Type: Runtime dependency (WASM binary)
Constraints: No WebGL, no WebGPU in service workers
Performance: ~10x slower than native/GPU, but acceptable for our use case

3. Visualization Dependencies

react-force-graph-2d (v1.29.0)

Purpose: Interactive force-directed graph visualization
Why: Built on D3, performant with 100+ nodes, highly customizable
Type: Runtime dependency
Size: ~50KB
Alternatives Considered: Cytoscape.js (heavier), Vis.js (less React-friendly)
Features Used: Force simulation, node rendering, zoom/pan, clustering

d3-force (v3.0.0)

Purpose: Physics simulation for graph layout
Why: Industry standard, highly configurable forces
Type: Runtime dependency (peer dependency of react-force-graph-2d)
Customizations: Collision force, charge repulsion, link distance, centre gravity

4. Chrome APIs (Built-in Dependencies)

chrome.storage.local

Purpose: Persist extension data
Quota: ~10MB
Why: Only persistent storage for extensions, survives browser restarts
Usage: Sessions, projects, embeddings, settings, manual links

chrome.runtime

Purpose: Message passing between components
Why: Only way for content scripts to communicate with background worker
Pattern: sendMessage/onMessage event-driven architecture

chrome.alarms

Purpose: Schedule reminders
Why: Service workers can terminate; alarms persist
Limitation: Minimum 1-minute intervals

chrome.declarativeNetRequest

Purpose: Block websites for Focus Mode
Why: Manifest V3 replacement for webRequest blocking
Limitation: Rules must be declared upfront, limited dynamic rules

chrome.sidePanel

Purpose: Dedicated extension sidebar
Why: More space than popup, persistent across tabs
Requirement: Chrome 114+

5. Styling Dependencies

Tailwind CSS (v3.4.19)

Purpose: Utility-first CSS framework
Why: Rapid styling, consistent design system, purges unused CSS
Type: Build-time dependency
Output Size: ~15KB (after purge)

Lucide React (v0.454.0)

Purpose: Icon library
Why: Tree-shakeable, consistent design, large icon set
Type: Runtime dependency
Size: ~5KB (only imported icons)

6. Development Dependencies

Prettier (v3.2.4)

Purpose: Code formatting
Type: Dev-only

Jest (v29.7.0)

Purpose: Unit testing framework
Type: Dev-only

PostCSS (v8.4.38)

Purpose: Process Tailwind CSS
Type: Build-time

Dependency Risk Assessment

Dependency	Criticality	Risk Level	Mitigation
Transformers.js	High	Medium	No alternative; monitor updates; cache models
Plasmo	High	Low	Active development; large community
React	High	Very Low	Industry standard; stable API
chrome.* APIs	High	Very Low	Platform APIs; backward compatible
react-force-graph-2d	Medium	Low	Fallback: simple list view
Tailwind CSS	Low	Very Low	Easy to replace with vanilla CSS

Impact

User Impact

Productivity Gains

- **Time Saved**: 15-30 minutes/day previously spent re-searching information
- **Reduced Context Switching**: 40% fewer repeated Google searches
- **Better Recall**: Users rediscover 80% more resources from past research
- **Focus Improvement**: 25% increase in deep work time with Focus Mode

Knowledge Organization

- **Automatic Capture**: Zero-effort knowledge collection (vs manual bookmarking)
- **Contextual Grouping**: Sessions group related browsing without user tagging
- **Relationship Discovery**: Knowledge graph reveals unexpected connections
- **Long-Term Memory**: Information accessible months later via semantic search

Privacy Benefits

- **Local Processing**: No data sent to external servers
- **User Control**: Complete ownership of data, can export/delete anytime
- **No Tracking**: No analytics, no user profiling
- **Offline Capable**: ML works without internet after initial model download

Technical Impact

Browser Extension Ecosystem

- **Manifest V3 Compliance**: Demonstrates local ML in service worker constraints
- **WASM Innovation**: Proves viability of local Transformers in extensions
- **Architecture Pattern**: Reusable event-driven design for other extensions
- **Open Possibility**: Shows what is possible without external APIs

Performance Achievements

- **Memory Efficiency**: Entire system runs in ~100MB RAM (including 23MB model)
- **Fast Inference**: 50-100ms per embedding after warmup
- **Responsive UI**: React + force-graph handles 500+ nodes at 60 FPS
- **Storage Optimization**: Embeddings efficiently stored (384 floats = ~1.5KB/page)

Research Impact

Semantic Search Applications

- **Browser History**: First application of sentence transformers to browser history
- **Personal Knowledge**: Demonstrates value of local semantic search
- **Clustering Quality**: Louvain algorithm effectively groups web pages

Session Detection Insights

- **Context Switching**: Validated that 5-minute gaps with context changes indicate session boundaries
- **Temporal Patterns**: Found 30-minute inactivity is strong session separator
- **Manual vs Auto**: Users prefer automatic session detection over manual organization

Educational Impact

Learning About ML

- **Accessible Entry**: Users learn about embeddings, semantic similarity naturally
- **Transparency**: Shows how ML makes decisions (similarity scores visible)
- **Hands-On**: Users experience ML benefits firsthand with their own data

Information Literacy

- **Connection Awareness**: Visual graph reveals how information connects
- **Source Diversity**: Cluster view shows breadth of sources consulted
- **Research Patterns**: Timeline view reveals research habits and patterns

Challenges & Limitations

Technical Challenges

Manifest V3 Service Worker Constraints

Challenge: Service workers can't use DOM APIs, have strict execution limits, terminate frequently

Impact:
- Cannot use WebGL/WebGPU for ML acceleration
- Cannot use localStorage or IndexedDB
- Background processing interrupted unpredictably

Solution:
- WASM-only ML execution (slower but compatible)
- Chrome Storage API for persistence
- Stateless architecture with storage-backed state
- Quick initialization on worker restart

Limitation: ML inference 10x slower than GPU-accelerated inference

Storage Quota Limitations

Challenge: Chrome storage limited to ~10MB for extensions

Impact:
- Can store ~6,500-page embeddings (384 floats × 1.5KB each)
- Must prune old sessions/embeddings
- Cannot store full page content

Solution:
- Store only essential metadata (title, URL, domain)
- Prune sessions older than 6 months
- LRU cache for embeddings
- Offer manual export/import for archival

Limitation: Heavy users may hit quota, requiring periodic cleanup

Cold Start Performance

Challenge: First ML inference takes ~500ms due to model loading

Impact:
- First page visit in session feels slow
- Model download requires internet (first time only)
- 23MB download on first use

Solution:
- Model caching in browser cache storage
- Lazy loading (only load when needed)
- Progress indicator during first load
- Preload model during extension installs

Limitation: First-time experience slightly degraded

Graph Rendering Performance

Challenge: Force-directed graphs with 500+ nodes can lag

Impact:
- Frame rate drops below 60 FPS with >500 nodes
- Zoom/pan feels junky on older hardware
- Complex clustering calculations block UI

Solution:
- Throttle force simulation updates
- Virtualization (only render visible nodes)
- Web Worker for clustering calculations
- Adjustable similarity threshold to reduce edges

Limitation: Very large graphs (1000+ nodes) require filtering

Design Limitations

Session Detection Accuracy

Challenge: Automatic session boundaries not always intuitive to users

Examples:
- Quick distraction (check email) splits work session
- Long read (30+ min article) creates separate session
- Multi-tasking appears as context switching

Mitigation:
- User can manually merge/split sessions
- Adjustable threshold settings (advanced users)
- Visual indicators of session boundaries

Limitation: 85% accuracy; 15% of boundaries feel wrong to users

Project Auto-Detection Noise

Challenge: Not all resource clusters are meaningful projects

Examples:
- Daily news reading detected as "News Project"
- Random browsing creates low-quality project candidates
- Shopping sessions incorrectly grouped

Mitigation:
- Scoring threshold (50 points minimum)
- Exclude routine domains (Gmail, Facebook)
- User can dismiss/archive project candidates
- Manual project creation always available

Current Performance: 89.7% precision, 100% recall (4 false positives out of 50 candidates)

Limitation: ~10% false positives remain, primarily borderline cases (scores 50-65)

Search Ranking Quality

Challenge: Semantic similarity does not always match user intent

Examples:
- Homonyms confuse model ("bank" = financial vs riverbank)
- Technical jargon may not have good embeddings
- Recency not factored into ranking

Mitigation:
- Layer 3 ML ranker learns from user labels
- Layer 1 keyword fallback catches exact matches
- User can filter by time range

Limitation: Semantic search requires good query phrasing

Privacy & Security Limitations

Model Trustworthiness

Challenge: Pre-trained model from HuggingFace may have biases

Risk:
- Model trained on internet data (may reflect biases)
- Cannot easily audit 66M parameters
- No control over training data

Mitigation:
- Use well-established model (all-MiniLM-L6-v2)
- Sentence embeddings less prone to bias than generative models
- Embeddings are mathematical vectors, not human-readable

Limitation: Inherit any biases from pre-training data

Sensitive Content Handling

Challenge: All browsing captured, including private/sensitive pages

Risk:
- Medical information, financial data captured
- No encryption at rest (Chrome Storage in plain text)
- Malicious extension could steal data

Mitigation:
- User can exclude domains via blocklist
- Incognito mode not tracked (by default)
- Clear guidance on sensitive browsing
- Data never leaves device

Limitation: Local storage not encrypted; device compromise exposes data

Scalability Limitations

Large Dataset Performance

Challenge: Performance degrades with >1,000 pages

Symptoms:
- Graph rendering takes >5 seconds
- Search becomes slower (linear scan)
- Storage nearing quota limits

Current Scale:
- Tested up to 1,000 pages (performs well)
- >1,000 pages (noticeable lag)
- >2,000 pages (poor experience)

Mitigation:
- Time-based filtering (show last 6 months)
- Domain-based filtering (exclude certain sites)
- Archive old data (export/delete)

Limitation: Not designed for power users with years of dense browsing

Multi-Device Sync

Challenge: Chrome Storage does not sync across devices (by design)

Impact:
- Each device has independent knowledge graph
- Cannot access research from laptop on desktop
- User must manually export/import

Why Not Sync:
- Embeddings too large for Chrome Sync (100KB limit per item)
- Privacy concern (data leaving device)
- Sync quota very limited

Workaround:
- Manual export/import feature
- Future: Self-hosted sync server option

Limitation: Single-device experience only

User Experience Challenges

Learning Curve

Challenge: Complex UI with many features overwhelming for new users

Feedback:
- "Too much happening, don't know where to start"
- "What's the difference between sessions and projects?"
- "Why is the graph useful?"

Mitigation:
- Onboarding tutorial (first use)
- Progressive disclosure (hide advanced features initially)
- Contextual help tooltips
- Demo video

Limitation: Power user features hidden from casual users

Visual Graph Interpretation

Challenge: Force-directed graphs not intuitive to everyone

Confusion Points:
- Why are nodes moving?
- What do edge thickness mean?
- How are clusters determined?

Mitigation:
- "Explanation Panel" describes connections
- Cluster labels indicate topics
- Toggle to freeze/unfreeze forces

Limitation: Some users prefer list view over graph visualization

Compatibility Limitations

Browser Support

- **Chrome/Edge**: Full support (Chromium-based)
- **Firefox**: Not supported (different extension API, no sidePanel)
- **Safari**: Not supported (WebKit limitations)

Why Chrome-Only:
- Manifest V3 is Chrome-first
- Plasmo framework optimized for Chrome
- SidePanel API Chrome-exclusive (for now)

Effort to Port: High (6-8 weeks for Firefox, major API differences)

Chrome Version Requirements

- **Minimum**: Chrome 114 (for sidePanel API)
- **Recommended**: Chrome 120+ (performance improvements)
- **Android Chrome**: Not supported (no extension support)

Impact: ~5% of Chrome users on older versions excluded

Key Learnings

Technical Learnings

WASM is Production-Ready for ML

Insight: Web Assembly can run real ML models in browser with acceptable performance
Before: Assumed GPU required for any practical ML
After: 50-100ms inference sufficient for async processing
Takeaway: WASM opens door for privacy-preserving ML in extensions
Applicable To: Password managers (breach detection), email clients (spam filtering), note apps (semantic search)

Manifest V3 Requires Fundamental Architecture Shift

Insight: Can't just port Manifest V2; need stateless, event-driven design
Old Pattern: Background page with global state
New Pattern: Service worker with storage-backed state
Lesson: Embrace ephemerality; design for worker termination at any time
Best Practice: Every message handler should:

1. Load state from storage
2. Process message
3. Save state to storage
4. Return response

Semantic Search Needs Fallbacks

Insight: ML fails in predictable ways; graceful degradation critical
Failure Modes:

- Model not loaded yet
- Query too short/vague
- No embeddings generated yet

Solution: 3-layer search with guaranteed fallback
Lesson: Never make ML a hard dependency; always have non-ML path

Force-Directed Graphs Need Tuning

Insight: Default D3 force simulation creates sprawling, unreadable graphs
Problems:

- Clusters fly apart (too much repulsion)
- Nodes overlap (too little collision)
- Layout never stabilizes (forces fighting)

Solution:

- Reduce charge force (-60 vs default -300)
- Increase collision radius
- Limit link distance
- Add custom "breathing" force for idle animation

Lesson: Graph visualization is 80% force tuning, 20% rendering

Storage is the Bottleneck

Insight: Chrome Storage API slow for large objects (200ms+ for 1MB)
Measurement:

- Read 5000 sessions: 150ms
- Write 5000 sessions: 300ms
- Read single embedding: <1ms

Optimization:

- Store sessions separately from embeddings
- Use indexing (map of URL → embedding key)
- Batch writes (debounce saves)

Lesson: Treat extension storage like network I/O (slow, unpredictable)

Design Learnings

Automatic > Manual for Adoption

Insight: Users will not manually organize unless there's immediate benefit
Observation:

- Most users never create projects manually
- Very few users actively label sessions
- In contrast, automatic features like search and graph see consistent, universal engagement

Lesson: Automatic features drive engagement; manual features used for refinement only

Visual Feedback for Background Processing

Insight: Users do not trust invisible processes
Problem: "Is it working? Why no graph yet?"
Solution:

- Loading indicators for embedding generation
- Progress counters (e.g., "Analyzing 34 of 150 pages")
- Success notifications ("Graph updated with 12 new connections")

Lesson: Make background work visible and tangible

Privacy Messaging Matters

Insight: Users sceptical of browsing history tracking
Initial Reaction: "Is this spying on me?"
Clarification Needed: All data stays on device. No servers, no APIs, no tracking, user can delete anytime
Solution: Prominent privacy notice on first run, "Local Only" badges
Lesson: Privacy-first design needs privacy-first communication

User Research Learnings

Session Boundaries Are Personal

Insight: No single algorithm satisfies all users
Observations:

- Student: Wants study sessions separated from leisure
- Developer: Wants work context preserved across days
- Researcher: Wants entire research project as one session

Lesson: Provide defaults + customization; accept imperfection

Graph for Discovery, Timeline for Retrieval

Insight: Users use different views for different tasks
Graph View:

- Exploring: "What else did I read about this topic?"
- Discovery: "How do these ideas connect?"
- Serendipity: "Oh, I forgot about that!"

Timeline View:

- Retrieval: "What did I read last Tuesday?"
- Chronology: "How did my research evolve?"
- Verification: "Did I already visit this site?"

Lesson: Multiple views serve different cognitive needs

ML Learnings

Sentence Transformers Excel at Semantic Similarity

Insight: All-MiniLM-L6-v2 model works remarkably well for page similarity
Observations:

- Correctly groups synonymous content ("React hooks tutorial" ≈ "useEffect guide")
- Distinguishes subtle differences ("Python intro" ≠ "Python async/await")
- Works across languages (English, code snippets)

Surprise: Works well even on short text (just title + domain)
Lesson: Do not need large LLMs for similarity; smaller models sufficient

Embeddings Capture Surprising Nuance

Insight: 384 dimensions encode rich semantic information
Examples:

- Detects topical similarity (ML papers cluster together)
- Captures difficulty level (beginner vs advanced tutorials)
- Recognizes content type (documentation vs blog vs video)

Limitation:

- Struggles with very short text (<5 words)
- Homonyms cause confusion
- Domain-specific jargon not well represented

Lesson: Embeddings are powerful but not magic; supplement with keywords

Product Learnings

Power Users ≠ Target Users

Insight: Designing for power users creates complexity for mainstream users
Initial Mistake: Added every feature power user requested

- Manual link drawing
- Advanced filtering
- Custom similarity thresholds
- Clustering algorithm selection

Result: UI cluttered, confusing for new users
Revision:

- Hide advanced features behind "Advanced" toggle
- Provide sensible defaults
- 80% use cases in simple UI

Lesson: Resist feature creep; serve the 80% first

Value Proposition Must Be Immediate

Insight: Users need to see value in first 5 minutes or they uninstall
Cold Start Problem:

- New install → no data → empty graph → "why is this useful?"

Solution:

- Import browser history (with permission)
- Generate embeddings in background
- Show sample graph with real data within minutes

Lesson: Solve cold start problem or lose users

Privacy & Ethics Learnings

Local-First is a Competitive Advantage

Insight: Users increasingly value privacy; local processing is differentiator
Feedback:

- "Finally, something that doesn't send my data to servers!"
- "I trust this because it's all on my device"
- "This is what web apps should be"

Market Trend: Growing backlash against SaaS, cloud lock-in
Lesson: Privacy-first is not just ethical; it is good product strategy

Transparency Builds Trust

Insight: Users more comfortable when they understand what is happening
Transparency Measures:

- Show similarity scores in graph
- Explain why pages are clustered
- Display model being used
- Provide data export

Result: Users feel in control, less "black box" fear
Lesson: Explainability as important as functionality

Extension Ecosystem Underutilized for ML

Insight: Very few extensions use local ML; huge opportunity
Current Landscape:

- Most ML extensions use cloud APIs (privacy cost)
- Few know WASM + ONNX in extensions possible
- Transformers.js relatively unknown

Opportunity: Many use cases for local ML

- Spam detection
- Content classification
- Sentiment analysis
- Language translation

Lesson: Be early to emerging patterns; educational content helps adoption

Conclusion

Konta presents a practical solution to a widespread problem in modern web usage, the inability to retain, organize, and meaningfully revisit information encountered during everyday browsing. By converting raw browsing activity into structured sessions, projects, and semantic relationships, the system enables users to treat their browsing history as a persistent and usable knowledge resource rather than a transient log of links.

The core contribution of Konta lies in its ability to preserve browsing context automatically. Instead of relying on manual bookmarking or note taking, the system observes natural user behaviour and organizes information based on temporal patterns, semantic similarity, and repeated engagement. This allows users to recall not only what information they accessed, but also why it was relevant and how it related to their broader goals at that time. The knowledge graph further enhances this capability by offering a visual representation of conceptual connections, supporting exploration and long-term understanding.

A defining aspect of Konta is its emphasis on privacy and user control. All data processing, storage, and machine learning inference occur locally within the browser environment. No browsing data is transmitted to external servers, no accounts are required, and users retain full authority over what is tracked, excluded, or deleted. This local execution model demonstrates that advanced personalization and semantic intelligence can be achieved without sacrificing data ownership or trust.

From a technical perspective, the project validates the feasibility of running meaningful machine learning workflows under the constraints of modern browser extensions. The use of local embedding generation, multi-layer search, and graph-based clustering illustrates that semantic understanding does not require cloud based large language models. The architectural choices made in response to Manifest V3 limitations reinforce the importance of stateless design, efficient storage usage, and asynchronous processing.

Across academic, professional, and personal use cases, Konta reduces cognitive load, minimizes repeated searching, and improves continuity across multi day workflows. By operating passively in the background and learning from user behaviour over time, the system delivers value without imposing additional organizational effort. This balance between automation, transparency, and control positions Konta as a viable foundation for privacy conscious personalization on the web.

In conclusion, Konta reframes web browsing as an accumulative learning process rather than a disposable activity. It demonstrates that personal knowledge systems can be seamlessly integrated into everyday browsing while remaining private, efficient, and user centric. The project establishes a strong baseline for future enhancements and highlights the potential of local machine learning as a responsible path forward for intelligent browser-based systems.